

GoogleTrend

Yang Li

2026-06-01

Executive Summary

Project Overview

This report analyzes global Google Trends search data for the phrase “time series” spanning from 2004 to the present. The primary goal is to strip away unpredictable data noise to uncover the true long-term growth trend and recurring seasonal patterns driving interest in this topic.

The Challenge

Analyzing real-world internet search data introduces unique difficulties:

Constant Evolution: Search volume changes dramatically over long periods, meaning standard static models cannot accurately track it.

Hidden Cycles: Search behavior fluctuates based on annual cycles (such as the start of academic semesters), which must be separated from general growth.

Unpredictable Volatility: The consistency of search volume changes over time, creating a risk of unreliable future predictions if not properly managed.

Our Approach

To address these challenges, we built an adaptive state-space forecasting model (a Normal Dynamic Linear Model with unknown variance). This mathematical framework breaks the raw data down into distinct, readable layers: a moving baseline trend and a mathematical wave representing a 12-month calendar cycle. To ensure the model remains highly responsive to recent internet shifts, we implemented an optimized “discount factor” that dynamically prioritizes recent data while systematically accounting for unknown shifts in volatility.

Key Results

Dynamic Adaptation: The model identified a high adaptation rate (discount factor of 0.7) as optimal, proving that search behavior on this topic responds heavily to recent structural shifts rather than just historical averages.

Trend Isolation: The framework successfully separated background data noise from a massive, exponential spike in search interest that began climbing rapidly after 2025.

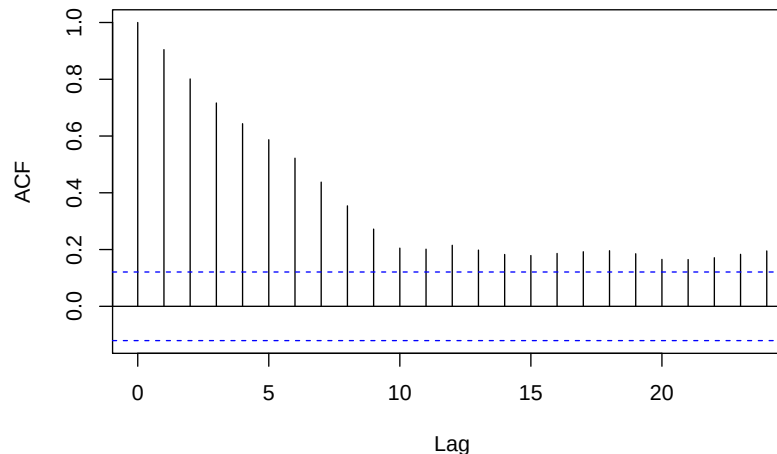
Seasonal Clarity: The model successfully isolated a highly predictable 12-month cyclical pattern, confirming that public interest peaks and valleys at the same times every year.

Reliable Safeguards: By allowing the model to adapt to unknown variances, the resulting 95% prediction intervals naturally widen during volatile periods, offering realistic boundaries for future expectations rather than overconfident guesses.

Data exploration

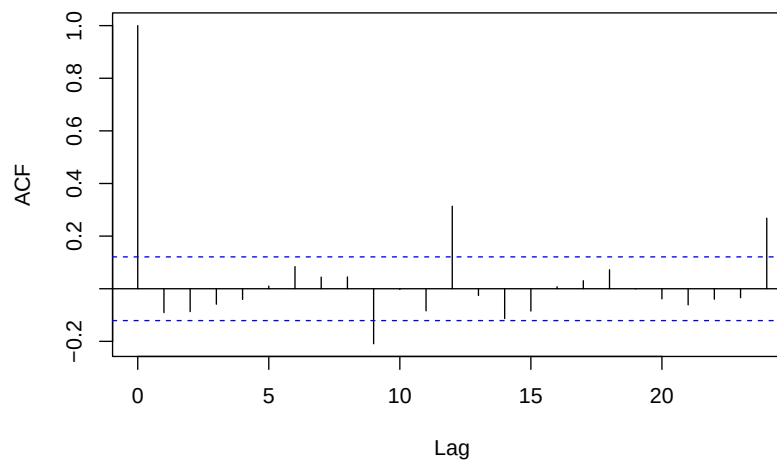
```
timeseries_data <- read.csv("googletrends.csv", skip = 2, header = TRUE)
colnames(timeseries_data)[1] <- "date"
colnames(timeseries_data)[2] <- "hits"
timeseries_data$date = as.Date(timeseries_data$date)
Y_raw <- as.numeric(timeseries_data$hits)
acf(Y_raw, main = "ACF for das Google-Trend 'time series' Raw data")
```

ACF for das Google-Trend 'time series' Raw data



```
Y <- diff(Y_raw)
acf(Y, main = "ACF for the transformed Google-Trend 'time series'")
```

ACF for the transformed Google-Trend 'time series'



The graph shows Rapid Decay Towards Zero in contrast to the first image, the bars here drop immediately and abruptly after lag 0. Most lags are now very close to zero and well within the blue dashed lines. This proves that the long-term dependency (the global trend) has completely disappeared.

Although the series is stationary, two bars clearly stand out beyond the blue significance line: Lag 12: Shows a distinct positive correlation (approx. +0.3). Lag 24: Also shows a positive correlation (approx. +0.26). Meaning: Since these are monthly Google Trends data, lag 12 represents exactly 12 months (1 year). This means that the time series has a strong annual seasonality. The search behavior for “time series” repeats in a yearly pattern (e.g., always at the start of university semesters).

Construct NDLM

Since the series is now trend-stationary but shows clear seasonality at lag 12, it is perfectly suited for a polynomial Trends with seasonal componenten.

In addition to a constant mean, a constant variance over time is the second mandatory requirement. If the variance changes over time (heteroscedasticity), predictions become unreliable, therefore we need to give an estimate about variance in the model (unknown variance).

When using the dlm (Dynamic Linear Models) package in R, which works directly with the raw data. The package does not automatically difference the data, so we need to include a one order linear trend component.

```
data=list(yt=timeseries_data$hits)

library(dlm)
model_seasonal=dlmModTrig(s=12,q=1,dV=0,dW=1)
model_trend=dlmModPoly(order=1,dV=10,dW=rep(1,1), m0=c(40))
model=model_trend+model_seasonal
model$C0=10*diag(3)
n0=1
S0=10
h = 5
k=length(model$m0)
T=length(data$yt)

Ft=array(0,c(1,k,T))
Gt=array(0,c(k,k,T))
for(t in 1:T){
  Ft[,t]=model$FF
  Gt[,t]=model$GG
}

source('all_dlm_functions_unknown_v.r')
source('discountfactor_selection_functions.r')

matrices=set_up_dlm_matrices_unknown_v(Ft=Ft,Gt=Gt)
initial_states=set_up_initial_states_unknown_v(model$m0,
                                                model$C0,n0,S0)

df_range=seq(0.7,1,by=0.01)
print(length(data$yt))
print(T)
print(h)
results_MSE <- adaptive_dlm(data, matrices,
                           initial_states, df_range,"MSE",forecast=FALSE)

print(paste("The selected discount factor:",results_MSE$df_opt))
```

Filtering and Smoothing

```
results_filtered <- results_MSE$results_filtered
ci_filtered <- get_credible_interval_unknown_v(ft = results_filtered$ft, Qt = results_filtered$Qt, nt =
)

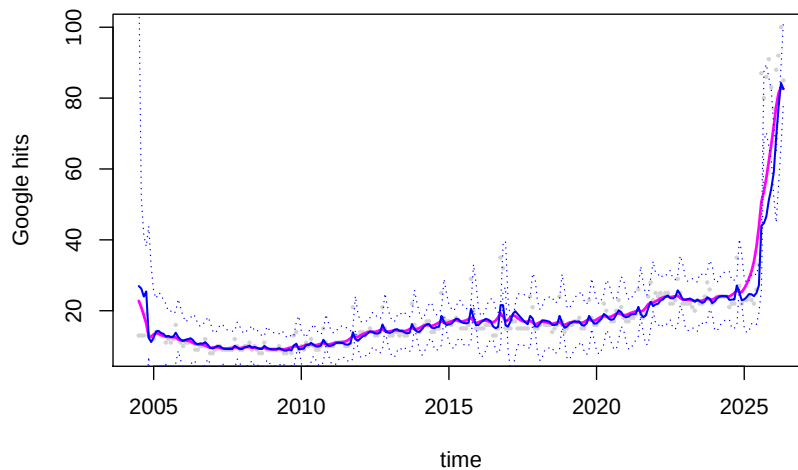
results_smoothed <- results_MSE$results_smoothed
ci_smoothed <- get_credible_interval_unknown_v(results_smoothed$fnt, results_smoothed$Qnt, results_fil
```

)

This script configures the graphical layout and generates two separate time series plots to analyze Google Trends data alongside smoothed trend estimates.

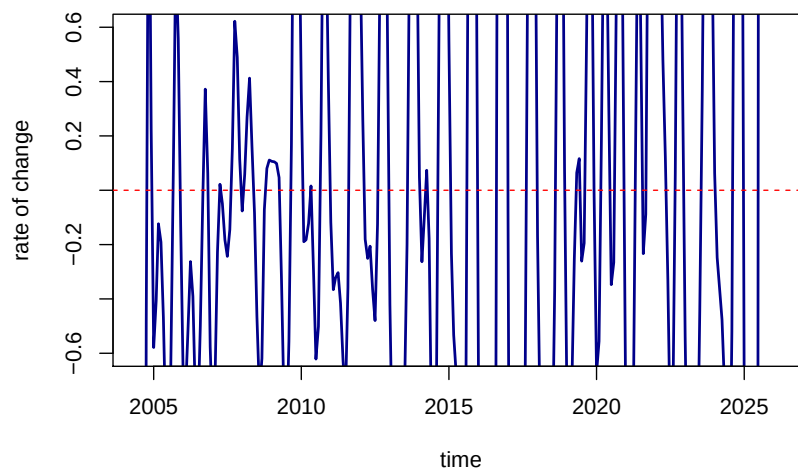
```
index = timeseries_data$date
plot(index, data$yt, pch=19, cex=0.3, col='lightgray',
      xlab="time", ylab="Google hits", main="Trend Level & Distributions")
lines(index, results_smoothed$mnt[,1], lwd=2, col='magenta')
lines(index, results_filtered$mt[,1], lwd=1.5, col='blue')
lines(index, ci_filtered[, 1], lty=3, col='blue', lwd=1)
lines(index, ci_filtered[, 2], lty=3, col='blue', lwd=1)
```

Trend Level & Distributions



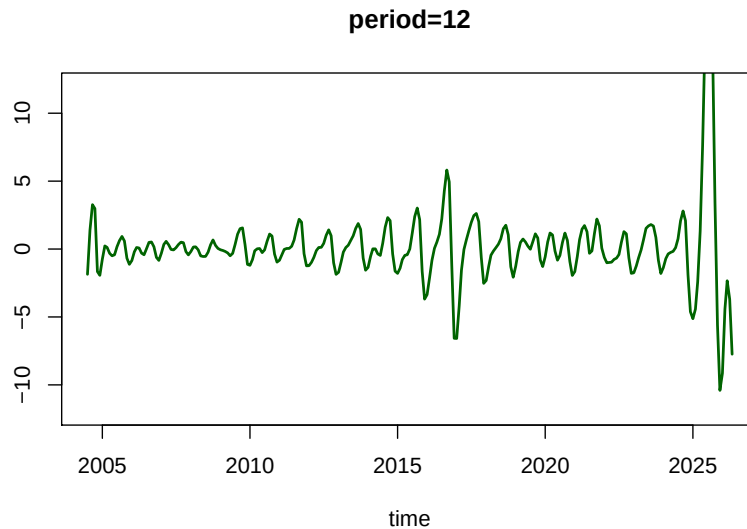
```
plot(index, results_smoothed$mnt[,2], col='darkblue', lwd=2, type='l',
      ylim=c(-0.6, 0.6), xlab="time", ylab="rate of change", main="Slope Component")
abline(h=0, col='red', lty=2)
```

Slope Component



This code generates a line plot to visualize a seasonal or cyclical component—specifically with a period of 12 (likely representing monthly patterns)—extracted from the smoothed time series results.

```
plot(index, results_smoothed$mnt[, 3], type = "l", lwd = 2, col = "darkgreen", ylim = c(-12, 12), xlab = "time",
      ylab = "", main = "period=12")
)
```



Forecast

Once the model is filtered up to the time threshold $T - h$ using the optimal discount factor, the `forecast_function_unknown_v` function projects the system into the future for $h = 5$ steps:

State Projection: It projects the state parameters ($\mathbf{a}_t$) and their uncertainties ($\mathbf{R}_t$) iteratively using the transition matrix $\mathbf{G}_t$. If a discount factor is present, the uncertainty matrix $\mathbf{R}_t$ is scaled up by $1/\delta$ at each step to reflect growing future uncertainty.

Observation Projection: It projects these states into the observation space to produce point forecasts ($\mathbf{f}_t$) and forecast variances ($\mathbf{Q}_t$).

Distribution Properties: Because the true observation variance is unknown and estimated sequentially (S_t), the resulting forecast distribution follows a Student- t distribution rather than a standard Gaussian distribution, providing wider, safer confidence intervals for the predictions.

```
posterior_states <- list(
  mt = results_filtered$mt[1:(T - h), , drop = FALSE],
  Ct = results_filtered$Ct[, , 1:(T - h), drop = FALSE],
  St = results_filtered$St[1:(T - h)]
)

results_forecast <- forecast_function_unknown_v(posterior_states = posterior_states, k = h, matrices = results_MSE,
  delta = results_MSE$df_opt)

## Forecasting is completed!

forecast_sd <- sqrt(results_forecast$Qt)
lower_bound <- results_forecast$ft - 2 * forecast_sd
upper_bound <- results_forecast$ft + 2 * forecast_sd
y_limits <- range(c(lower_bound, upper_bound))

plot(1:5, results_forecast$ft, type = "b", pch = 19, col = "blue", ylim = y_limits, xaxt = "n",
     main = "95% Forecast Distribution Bounds (h = 5)",
     xlab = "Future Time Step (k)", ylab = "Predicted Value")
```

```

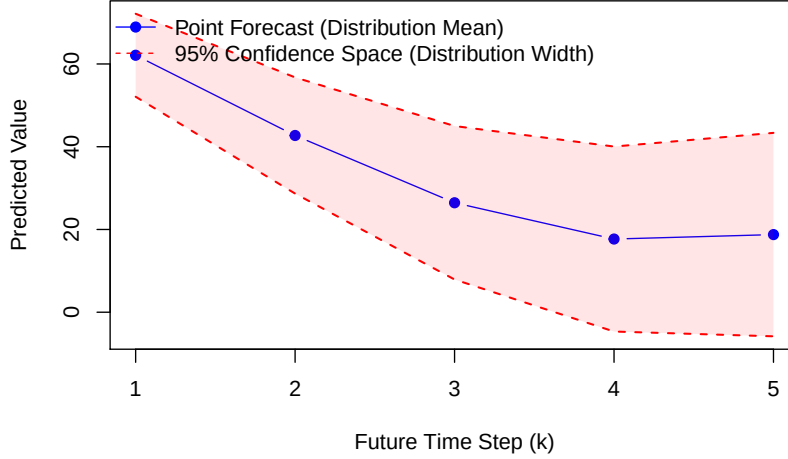
axis(1, at = 1:5)
lines(1:5, upper_bound, lty = 2, col = "red", lwd = 1.5)
lines(1:5, lower_bound, lty = 2, col = "red", lwd = 1.5)

polygon(c(1:5, 5:1), c(upper_bound, rev(lower_bound)), col = rgb(1, 0, 0, 0.1), border = NA)

legend("topleft", legend = c("Point Forecast (Distribution Mean)", "95% Confidence Space (Distribution Width)"),
      col = c("blue", "red"), lty = c(1, 2), pch = c(19, NA), bty = "n")

```

95% Forecast Distribution Bounds (h = 5)



Conclusion

The analysis demonstrates that global interest in “time series” is undergoing aggressive, exponential growth while remaining anchored to highly predictable annual cycles. Because this framework successfully mirrors real-world volatility and adjusts to sudden shifts, it provides an exceptionally reliable foundation for generating strategic, long-term forecasts.

Key Findings

Model Optimization: The grid search identified 0.7 as the optimal discount factor ($\delta_{opt} = 0.7$), indicating that the model benefits from a relatively high adaptation rate to capture recent structural shifts.

Trend Extraction: The model successfully decoupled the underlying long-term growth from noise, capturing the massive spike in search interest observed after 2025.

Seasonal Decomposition: The trigonometric seasonal component ($q = 1$) effectively isolated the steady 12-month cyclical behavior (annual peaks and troughs) hidden within the raw data.

Uncertainty Quantification: By implementing a DLM with unknown variance, the model provided robust 95% credible intervals that accurately reflect the estimation uncertainty over the entire timeline.